



第八章 异常处理

赵晓航

xiaohangzhao@mail.shufe.edu.cn

上海财经大学·信息管理与工程学院



目录

1. 异常基本概念
2. 异常处理结构
3. 主动抛出异常
4. 断言
5. 上下文管理
6. 编程实践
7. 本章习题



1. 异常基本概念

- **当程序运行出现错误时，就会产生异常（Exception）**
 - 例如除以0、下标越界、文件不存在、网络异常、类型错误、磁盘空间不足等
 - 软件的发展历史告诉我们：**异常是不可避免的**
 - 因此，正视异常，允许异常的产生，当异常产生了，采取适当措施处理异常即可
- **常见的Python异常类型**
 - **ValueError**：当函数接收到一个具有正确类型但不适当值的参数时触发
 - **TypeError**：当操作或函数应用于不适当类型的对象时触发
 - **IndexError**：当序列中没有此索引（index）时触发
 - **KeyError**：当字典中不存在指定的键时触发
 - **ZeroDivisionError**：当除数为零时触发
 - **FileNotFoundError**：当尝试访问不存在的文件时触发



- **例1：尝试访问不存在的字典键会触发KeyError：**

- `my_dict = {'a': 1, 'b': 2}`
- `print(my_dict['c']) # KeyError: 'c'`
- 通过检查键是否存在于字典中，或使用`dict.get()`方法，可以避免这种错误

- **不适当的类型会触发TypeError：**

- `name = 'Student NO.' + 1`
- `#TypeError: can only concatenate str (not "int") to str`
- 修改为：`name = 'Student No.' + str(1)`



2. 异常处理结构

- **语法异常**：在调试（debug）程序过程中即可发现错误，根据系统的错误提示改正即可
- **异常处理流程**：但有些错误却在调试时不易发现，比如网络异常、磁盘空间不足、下标越界等，只在某些特定条件下才可能发生。
 - 通过预先设计合适的异常处理机制，程序在遇到异常情况时能够自动执行这些处理流程，从而有效地纠正或管理这些异常
 - 这种做法不仅提高了程序的稳定性，而且确保了在遭遇意外情况时程序仍能平稳运行或优雅地终止



异常处理结构

• 异常处理结构

- 在Python编程中，处理异常的基础结构是使用`try...except...语句`。这种结构使得开发者能够捕捉并妥善处理在程序执行过程中可能出现的异常，从而防止程序意外崩溃
- 基本语法结构如下：
 - `try:`
 - `# 这部分是可能出现异常的代码块`
 - `except Exception [as reason]:`
 - `# 这部分是当捕捉到异常时执行的代码块，reason变量用于获取异常的具体信息`
 - 注：`except Exception` 可以用来捕获任意类型的异常

• 异常处理的逻辑

- 如果`try`代码块中的语句运行无异常：程序将跳过`except`块，继续执行后续代码
- 如果`try`代码块中出现异常，且该异常被`except`子句捕获：那么程序将执行`except`子句中的异常处理代码
- 如果`try`代码块中出现异常，但未被`except`捕获：则异常会被向外抛出
- 如果所有层次都没有捕获和处理该异常：程序最终将终止，并向用户显示异常信息



- **例:**

- while True:
- try:
- x = int(input("Please enter a number: "))
- break
- except ValueError:
- print("This is not a valid number. Please try again...")
- 如果用户输入的不是数字, 将产生ValueError异常



扩展的异常处理结构

- 除了基本的try...except...结构外，还可以加入else和finally语句，以及多个except子句，形成更完整的异常处理流程
- **扩展的语法结构**
 - try:
 - # 可能引发异常的代码
 - except Exception1 [as reason1]:
 - # 处理第一种类型的异常
 - except Exception2 [as reason2]:
 - # 处理第二种类型的异常
 - ...
 - else:
 - # 如果没有异常发生，则执行这个代码块
 - finally:
 - # 无论是否发生异常，都会执行这个代码块，通常用于释放资源



- 例:

- try:
- # 尝试执行的代码, 可能会产生异常
- result = 10 / 0
- except ZeroDivisionError:
- # 处理除零异常
- print("不能除以零。")
- except TypeError:
- # 处理类型错误异常
- print("类型错误。")
- else:
- # 如果没有异常发生
- print("操作成功。")
- finally:
- # 无论是否发生异常
- print("操作完成。")



3. 主动抛出异常

- **raise语句**：允许程序员强制触发指定的异常，这在需要明确指出错误条件或强制执行特定错误处理逻辑时非常有用
- **基本语法结构**：
 - `raise ExceptionType("错误信息")`
 - `ExceptionType`是要抛出的异常类型，如`ValueError`, `TypeError`等，而“错误信息”则是提供给异常处理器的额外信息



举例：数据通信

- 在数据通信中，**确保所有的操作都在有效连接上执行非常关键**。如果尝试在尚未建立或已关闭的连接上发送数据，可能会导致程序错误或数据丢失。
- 为了管理这种潜在的错误并增强程序的健壮性，开发者可以**采用异常处理机制来明确地监控和响应连接状态**
 - `def send_data(data, connection):`
 - `if not connection.is_open():`
 - `raise ConnectionError("Connection is not open.")`
 - `connection.send(data)`
 - `connection = NetworkConnection()`
 - `try:`
 - `send_data("Hello", connection)`
 - `except ConnectionError as e:`
 - `print(e) # 输出: Connection is not open.`
 - `send_data`函数在发送数据前检查网络连接是否开启，如果连接没有开启，则抛出`ConnectionError`



两种错误管理策略

- **两种常见的错误管理策略：**使用预检查和抛出异常处理

- **预检查方法：**

- `send_data(data, connection):`
- `if not connection.is_open():`
- `print("Connection is not open.")`
- `return`
- `connection.send(data)`
- `connection = NetworkConnection()`
- `send_data("Hello", connection)`

- **优点：**

- 直观和显式：代码中直接检查条件，如连接状态，可以让代码的逻辑更加明确和直观
- 防御性编程：通过预检查防止错误的发生，可以减少异常的使用，使得程序结构更加简单

- **缺点：**

- 代码冗余：如果多个地方需要做同样的检查，可能导致代码重复
- 灵活性差：预检查方法通常针对特定的错误条件，如果错误类型或检查逻辑需要变化，可能需要更多的代码修改
- 错误处理分散：如果检查逻辑散布在代码的多个部分，那么维护错误处理逻辑可能变得复杂



- **抛出异常方法:**

- **优点:**

- **集中错误处理:** 通过异常可以将错误处理逻辑集中在一个或几个地方, 便于维护和修改。
 - **处理意外情况:** 异常处理非常适合处理那些难以预料的错误或程序中的非正常情况。
 - **强制错误处理:** 使用异常可以强制调用者处理潜在的错误, 特别是在API和库设计中非常有用。

- **缺点:**

- **可能隐藏错误:** 过度依赖异常可能导致开发者忽视防御性编程, 从而使得某些错误在发生前未能被充分考虑。

- **结论:**

- 如果**错误条件预期会频繁发生**, 或者**错误检查是程序正常流程的一部分**, 使用**预检查**可能更合适。
 - 如果**错误情况较少**, 或者**错误处理需要集中管理**, 使用**异常抛出**可能更有优势。
 - 最佳实践是结合使用这两种方法: 在可能的情况下进行预检查来避免错误, 同时通过异常处理来管理不可避免或未预料到的错误情况



自定义异常类

- **自定义异常类**: 用于表示程序中的特定错误情况, 并使用raise语句主动抛出
 - 自定义异常通常继承自内置的异常类, 比如Exception
- **定义自定义异常**
 - `class MyCustomError(Exception):`
 - `def __init__(self, message):`
 - `self.message = message`
 - `super().__init__(message)`
- **使用raise抛出自定义异常**
 - `def check_age(age):`
 - `if age < 18:`
 - `raise MyCustomError("年龄不足18岁")`
 - `return "年龄已确认"`
 - `try:`
 - `user_age = check_age(16)`
 - `except MyCustomError as e:`
 - `print("发生异常: ", e)`



4. 断言

- **断言**：是一种在软件开发过程中提供一种**快速的错误识别机制**.
 - 通过在代码中插入断言，可声明某个条件必须为真，从而检查程序的内部状态或者变量的值。如果断言的条件计算结果为假，程序将抛出一个AssertionError，提示在这一点上存在问题
 - 断言assert常用于程序调试
- **assert语法结构**
 - `assert expression[, reason]`
- 当程序处于**调试 (debug)** 状态，即设置__debug__为True时，assert语句才会被执行；否则，即程序处于**发布 (release)** 状态，该语句不会被执行，从而提高程序运行速度
- **断言与异常处理经常结合使用**
 - **断言可被视为一种内部检查机制**
 - **异常处理则用于处理程序与外部交互时可能遇到的错误情况**



- 举例:
 - `def get_first_element(data_list):`
 - `assert len(data_list) > 0, "The list cannot be empty!"`
 - `return data_list[0]`
 - `data = [1, 2, 3, 4, 5]`
 - `try:`
 - `print("First element:", get_first_element(data))`
 - `except AssertionError as error:`
 - `print("Error:", error)`
 - `try:`
 - `empty_data = []`
 - `print("First element:", get_first_element(empty_data))`
 - `except AssertionError as error:`
 - `print("Error:", error)`



上下文管理

- **上下文管理**：主要用于简化资源的管理，特别是在那些需要精确控制资源释放的场景中
 - 广泛应用于网络通信、数据库连接、文件操作、多线程和多进程等领域
 - **with语句的优势在于**，它能够保证即使在发生异常的情况下，资源也能被适当地释放和清理
- **with语句的基本语法结构**
 - `with context_expression [as var]:`
 - `with`代码块
 - `context_expression`通常是一个上下文管理器，而可选的`as var`部分则是将这个管理器返回的对象赋值给变量



举例：上下文管理

- **举例：**以数据库连接为例，使用with语句可以确保数据库会话在使用后正确关闭，即使在数据库操作过程中发生异常也能保证这一点
 - `import sqlite3`
 - `# 假设数据库文件为'database.db'`
 - `with sqlite3.connect('database.db') as conn:`
 - `cursor = conn.cursor()`
 - `cursor.execute('SELECT * FROM some_table')`
 - `for row in cursor.fetchall():`
 - `print(row)`
 - with语句用于建立与SQLite数据库的连接，并将连接对象赋值给变量conn。with代码块内部的代码用于执行数据库查询并处理结果。当with代码块执行完毕后，无论是正常结束还是由于异常，数据库连接都会自动关闭



6. 编程实践

- **例1：从文件中读取指定行数的文本内容**
 - 首先将指定的文本内容写入文件，然后根据用户输入，从文件中读取指定行数。如果用户输入的行数大于文件中的行数，程序将读取并输出文件的所有内容。该程序还需要处理可能的异常情况，包括文件写入时的 I/O 错误、文件不存在错误，以及用户输入无效数字的情况。



```
def write_to_file(file_path, file_content):
    try:
        with open(file_path, "w") as file:
            file.write("\n".join(file_content))
        print(f"已成功将内容写入文件 '{file_path}'。")
    except IOError as e:
        print(f"错误：无法写入文件 '{file_path}'。发生了 I/O 错误：{e}")
    except Exception as e:
        print(f"发生了未知错误：{e}")

def read_file_lines(file_path, num_lines):
    try:
        with open(file_path, "r") as file:
            lines = file.readlines()
            if num_lines > len(lines):
                num_lines = len(lines)
            print(f"文件只有 {len(lines)} 行，显示所有内容：")
        else:
            print(f"文件的前 {num_lines} 行内容如下：")
            for i in range(num_lines):
                print(lines[i].strip())
    except FileNotFoundError:
        print(f"错误：文件 '{file_path}' 不存在。")
    except Exception as e:
        print(f"发生了一个未知错误：{e}")
```

```
file_path = "example.txt"
file_content = [
    "空山新雨后，天气晚来秋。",
    "明月松间照，清泉石上流。",
    "竹喧归浣女，莲动下渔舟。",
    "随意春芳歇，王孙自可留。"]

write_to_file(file_path, file_content)

try:
    num_lines = int(input("请输入要读取的行数："))
    read_file_lines(file_path, num_lines)
except ValueError:
    print("错误：请输入有效的数字。")
```



- 运行结果1（如果输入数字2）：
 - 已成功将内容写入文件 'example.txt'。
 - 请输入要读取的行数： 2
 - 文件的前 2 行内容如下：
 - 空山新雨后，天气晚来秋。
 - 明月松间照，清泉石上流。
- 运行结果2（如果输入单词two）
 - 已成功将内容写入文件 'example.txt'。
 - 请输入要读取的行数： two
 - 错误：请输入有效的数字。



- **例2：模拟一个银行账户的取款操作**，使用异常处理来防止取款金额超过账户余额、无效的取款金额（例如负数或非数字值）和数据库操作异常（如果涉及到数据库交互）等情况。

```
class InvalidAmountError(Exception):
    pass

class DatabaseError(Exception):
    pass

class InsufficientFundsError(Exception):
    pass

class BankAccount:
    def __init__(self, initial_balance):
        self.balance = initial_balance

    def withdraw(self, amount):
        # 检查是否为有效金额
        if not isinstance(amount, (int, float)) or amount <= 0:
            raise InvalidAmountError(f'无效的取款金额: {amount}')

        # 模拟数据库操作异常
        if amount == 12345:
            raise DatabaseError("数据库操作异常")

        if amount > self.balance:
            raise InsufficientFundsError(f'账户余额不足, 当前余额为: {self.balance}, 尝试取款: {amount}')

        self.balance -= amount
        return self.balance
```

```
# 创建一个账户
account = BankAccount(10000)

# 尝试各种取款操作
def attempt_withdrawal(amount):
    try:
        print(f"尝试取款: {amount}")
        print(f"取款后余额: {account.withdraw(amount)}")
    except InsufficientFundsError as e:
        print("发生错误: ", e)
    except InvalidAmountError as e:
        print("发生错误: ", e)
    except DatabaseError as e:
        print("发生错误: ", e)

# 尝试各种取款情况
attempt_withdrawal(15000) # 超出余额
attempt_withdrawal(-100) # 无效金额
attempt_withdrawal('abc') # 无效金额
attempt_withdrawal(12345) # 数据库异常
attempt_withdrawal(5000) # 正常取款
```



7. 本章习题

1. 断言 (assert) 在Python中通常用于什么目的？它与异常处理有何不同？
2. 编写一个函数，接受用户输入的数字并返回其平方。使用try...except块来处理可能出现的任何错误，并给出相应的错误提示
3. 编写一个函数，使用断言来确保输入的参数是正数。
4. 编写一个使用with语句的程序，该程序应打开一个文件并读取内容，同时确保在发生任何异常时文件能被正确关闭。
5. 创建一个名为OutOfRangeError的自定义异常，然后编写一个函数，如果输入的数字不在指定范围内（例如1到10），则抛出这个异常。
6. 编写一个程序，要求用户输入一个文件名，然后将用户输入的内容写入该文件中，直到用户输入 “exit” 停止输入。程序需要捕获以下异常：
 - 文件写入异常 (IOError) 并输出 “写入失败”。当指定的文件路径不存在或不可写入时，触发该异常。
 - 捕获用户未输入文件名的情况，并输出 “请输入有效的文件名”。



The End !